

Advanced Machine Learning

Gradient-based Learning & Backpropagation

Lukas Galke Poech

Spring 2026

Keywords

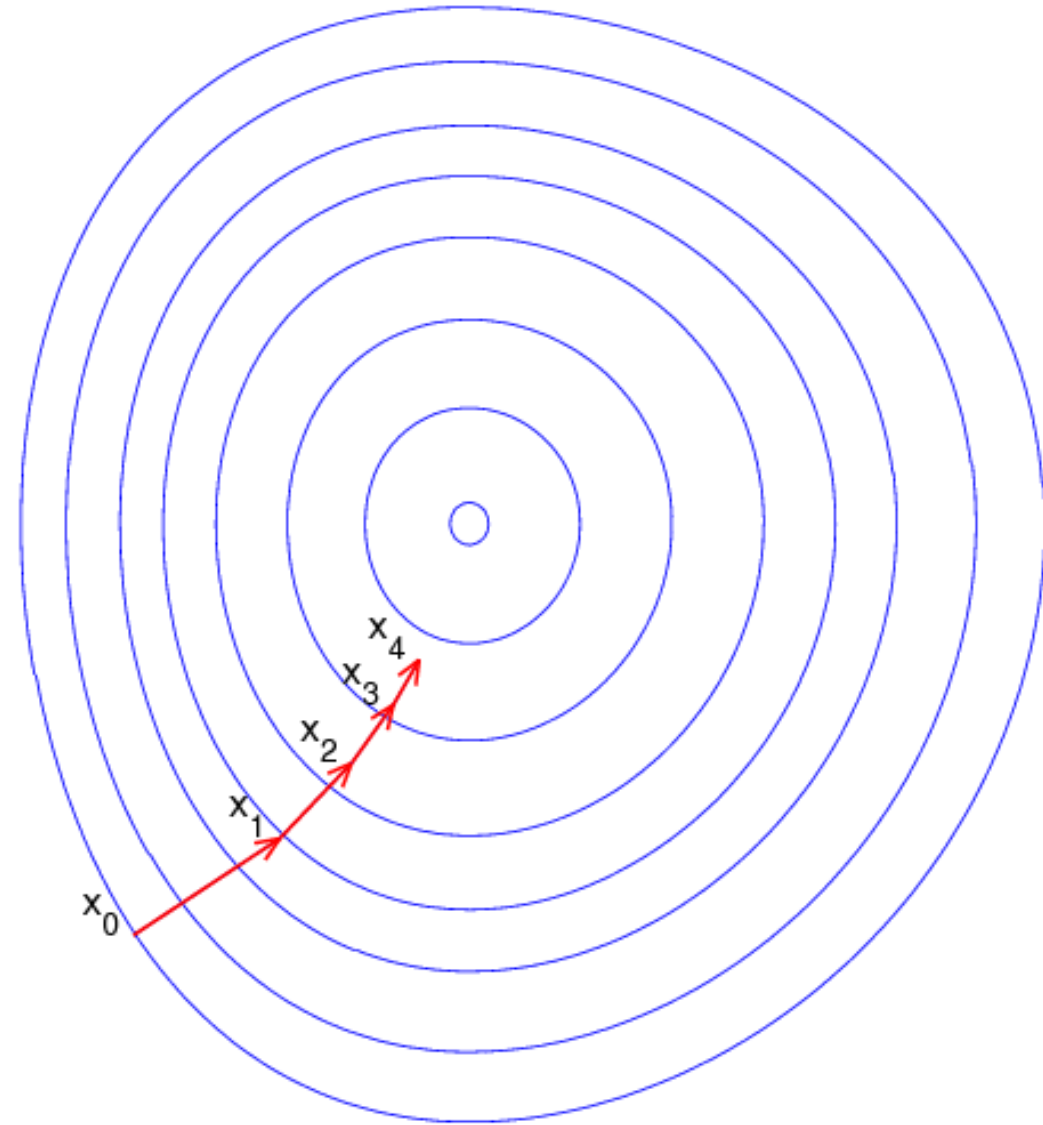
- **Gradient-based learning**
- Stochastic gradient descent
- Challenges of optimization
- Backpropagation

Gradient Based Learning



The Central Idea

- Update the model parameters following the steepest slope



More Mathematically

- Suppose function $y = f(x)$
- Derivative of function denoted: $f'(x)$ or as dy/dx
 - Derivative $f'(x)$ gives the slope of $f(x)$ at point x
 - It specifies how to scale a small change in input to obtain a corresponding change in the output:

$$f(x + \varepsilon) \approx f(x) + \varepsilon f'(x)$$

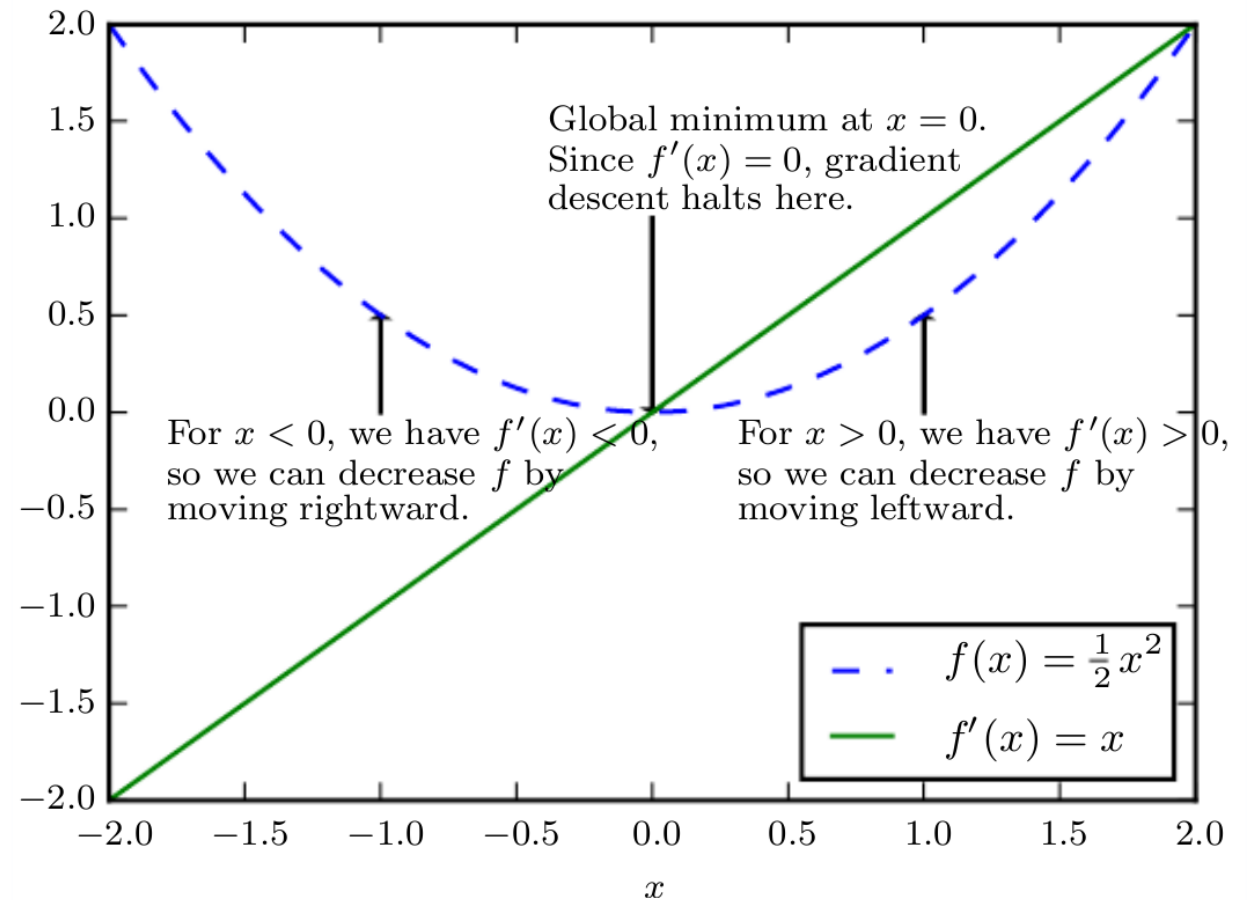
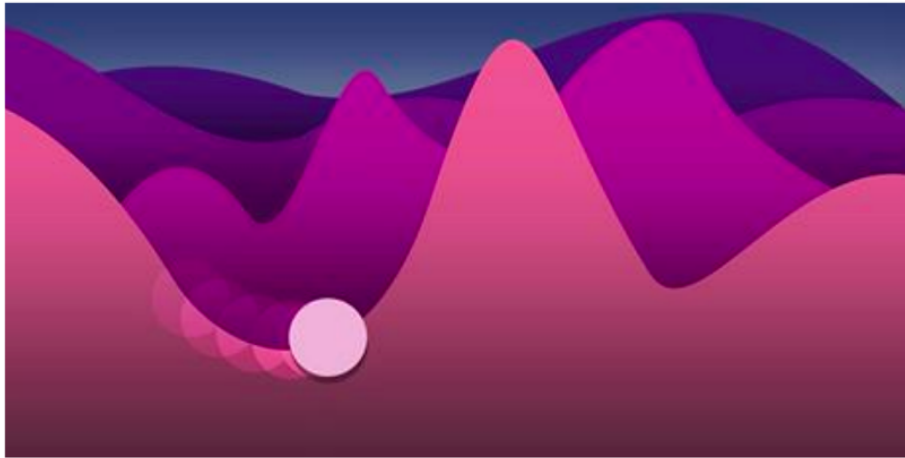
- We know that

$$f(x - \varepsilon \text{sign}(f'(x)))$$

is less than $f(x)$ for small ε .

- Thus we can reduce $f(x)$ by moving x in small steps with opposite sign of derivative
- This technique is called gradient descent (Cauchy 1847)

Usage of Derivates



Functions with multiple inputs

- Need partial derivatives

$$\frac{\partial}{\partial x_i} f(\mathbf{x})$$

- Measures how f changes as only variable x_i increases at point $\mathbf{x}$
- Gradient is vector containing all of the partial derivatives denoted with $\nabla_{\mathbf{x}} f(\mathbf{x})$
 - Element i of the gradient is the partial derivative of f wrt x_i
 - Critical points are where every element of the gradient is equal to zero
 - A function can be minimized when moving in the direction opposite to the gradient

Functions with multiple inputs

- We can decrease f by moving in the direction of the negative gradient vector
- Steepest descent proposes a new point

$$\mathbf{x}' = \mathbf{x} - \epsilon \nabla_{\mathbf{x}} f(\mathbf{x})$$

- With ϵ being the learning rate (there are many methods of defining ϵ)
- Ascending an objective function of discrete parameters is called hill climbing

Keywords

- Gradient-based learning
- **Stochastic gradient descent**
- Challenges of optimization
- Backpropagation

Stochastic Gradient Descent (SGD)

- A recurring problem in machine learning:
 - large training sets are necessary for good generalization
 - but large training sets are also computationally expensive
- Nearly all deep learning is powered by one very important algorithm: **Stochastic Gradient Descent**

Insight of SGD

➤ Insight: Gradient descent based on only a sample (we don't have the universe as data) is an **expectation**

➤ Expectation may be approximated using small set of samples

➤ In each step of SGD we can sample a minibatch of examples

$$B = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m')}\}$$

➤ drawn uniformly from the training set

➤ Minibatch size m' is typically chosen to be small: 1 to a hundred

➤ Crucially m' is held fixed even if sample set is in billions

➤ We may fit a training set with billions of examples using updates computed on only a hundred examples

SGD Estimate on minibatch

➤ Estimate of gradient is formed as

$$\mathbf{g} = \frac{1}{m'} \nabla_{\theta} \sum_{i=1}^{m'} L(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$

using only the examples of the minibatch

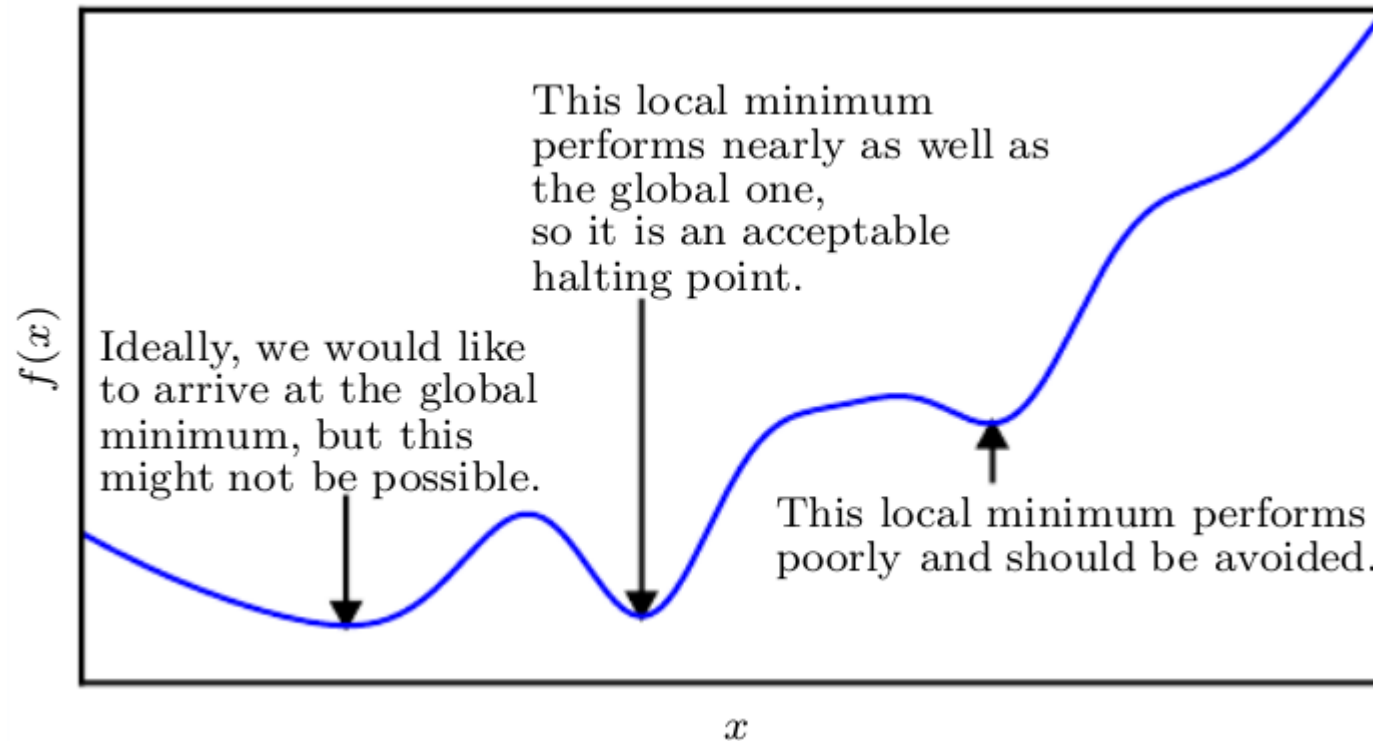
➤ SDG then simply follows the estimated gradient downhill

$$\theta' = \theta - \epsilon \mathbf{g}$$

How good is SGD?

- In the past gradient descent was regarded as slow and unreliable
- Application of gradient descent to non-convex optimization problems was regarded as unprincipled
- SGD is not guaranteed to arrive at even a local minimum in reasonable time
- **But it often finds a very low value of the cost function quickly enough**
- As $m \rightarrow \infty$ the model will eventually converge to its best possible test error before SGD has sampled every example

Good Enough in Practice



Keywords

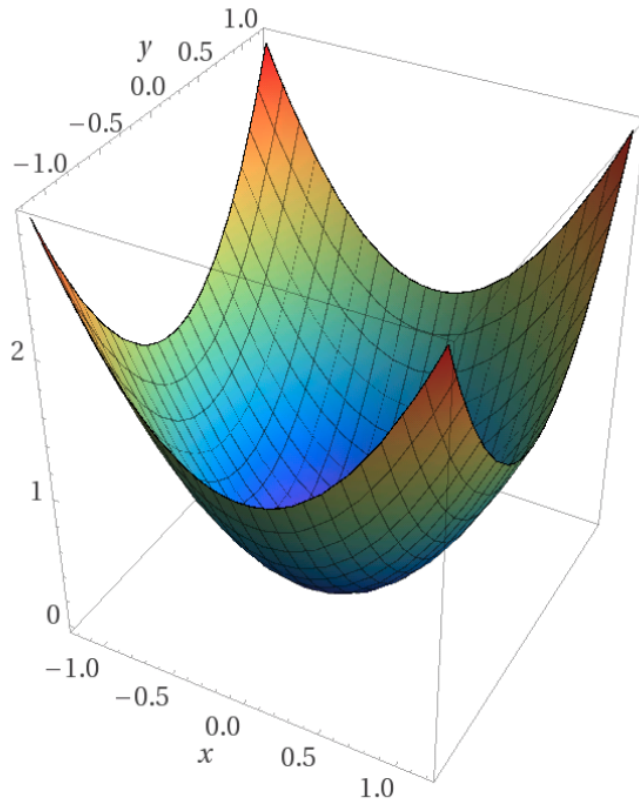
- Gradient-based learning
- Stochastic gradient descent
- **Challenges of optimization**
- Backpropagation

Specialties of Deep Learning

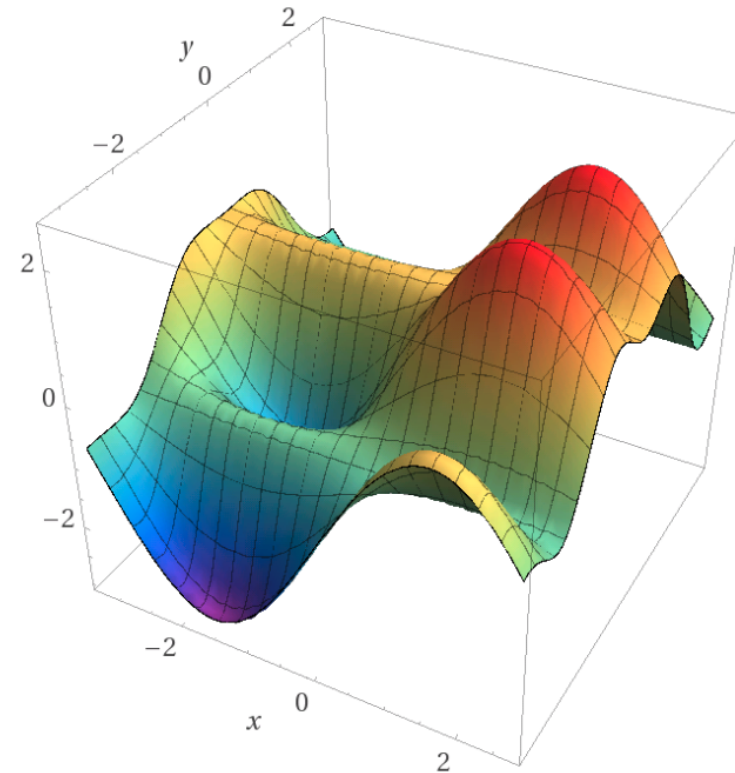
- Neural Network training not different from ML models with gradient descent. The components are needed:
 1. optimization procedure, e.g., gradient descent
 2. cost function, e.g., MLE
 3. model family, e.g., linear with basis functions

- Difference: **nonlinearity causes non-convex loss**
 - Use iterative gradient-based optimizers that merely drives cost to low value
 - No guarantees in comparison to convex optimizations
 - The initialization matters

Convex vs. Non-Convex



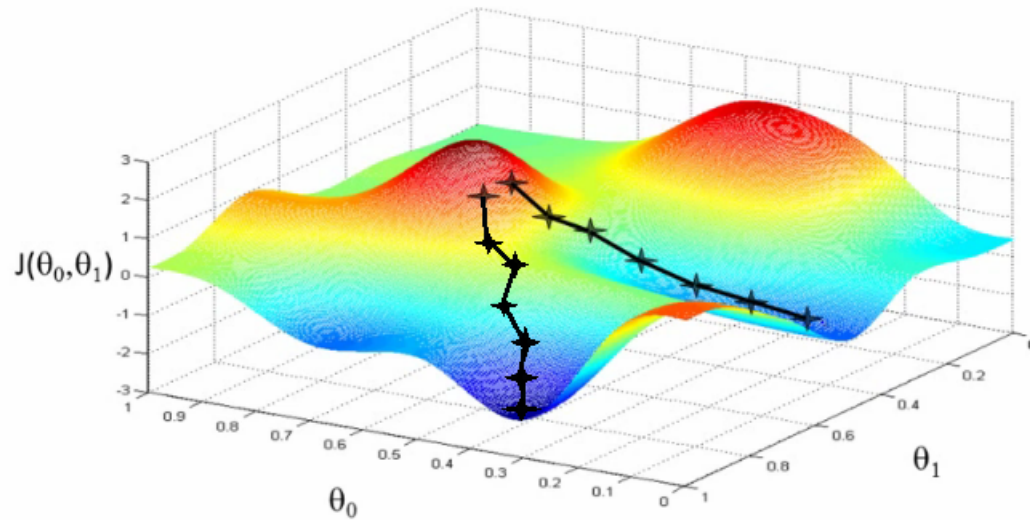
Computed by Wolfram|Alpha



Computed by Wolfram|Alpha

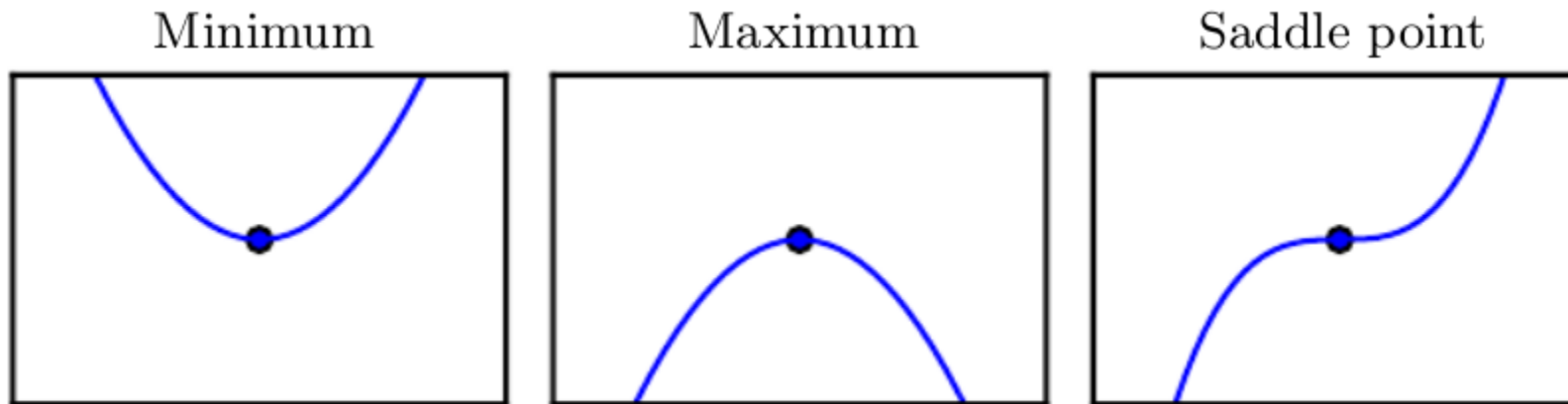
> <https://www.matroid.com/blog/post/the-hard-thing-about-deep-learning>

Problem: We can end-up in local minima



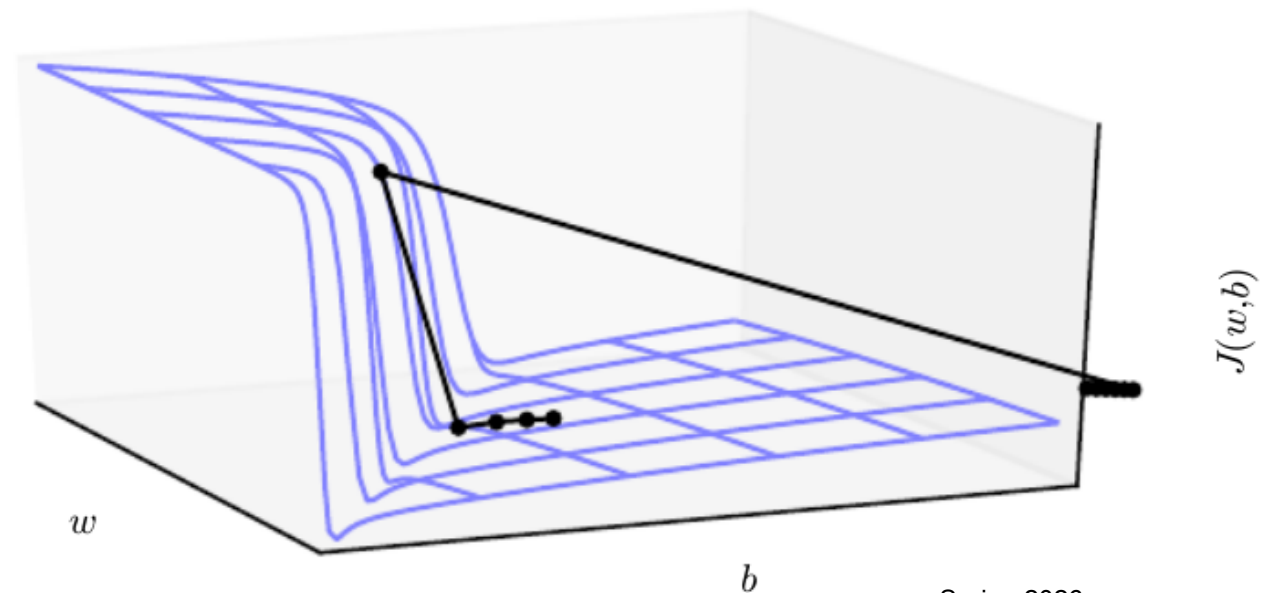
Problem: Stationary points, Local Optima

- When $f'(x) = 0$ derivative provides no information about direction of move
- Points where $f'(x) = 0$ are known as stationary or critical points
 - **Local minimum/maximum**: a point where $f(x)$ lower/ higher than all its neighbors
 - **Saddle Points**: neither maxima nor minima



Cliffs and Exploding Gradients

- Neural networks with many layers have steep regions i.e., cliffs
 - Result from multiplying several large weights
 - E.g., RNNs with many factors at each time step
- Gradient update step can move parameters extremely far, jumping off cliff altogether
- Cliffs dangerous from either direction
- Gradient clipping heuristics can be used



Inexact Gradients

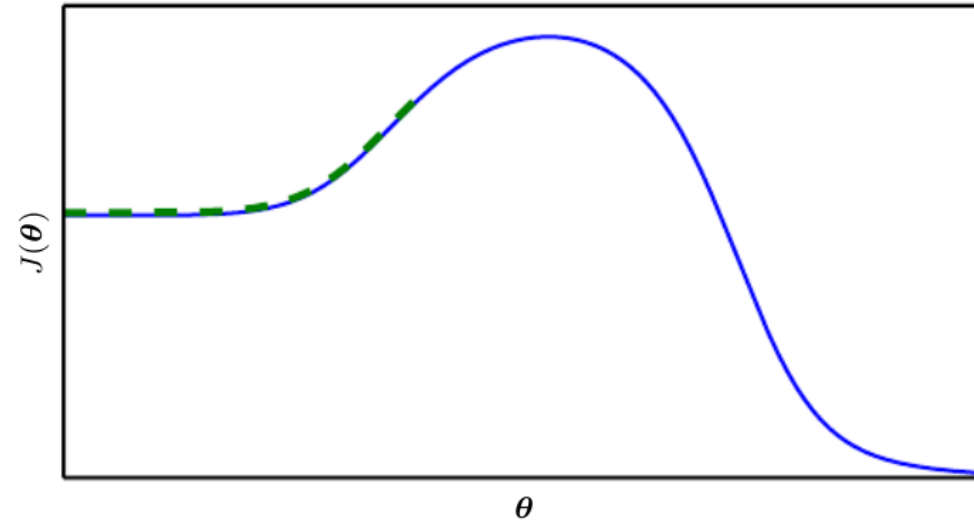
- Optimization algorithms assume we have access to exact gradient or Hessian matrix
- In practice we have a noisy or biased estimate
 - Every deep learning algorithm relies on sampling-based estimates
 - In using minibatch of training examples
 - In other case, objective function is intractable
 - In which case gradient is intractable as well

Poor Correspondence between Local and Global Structure

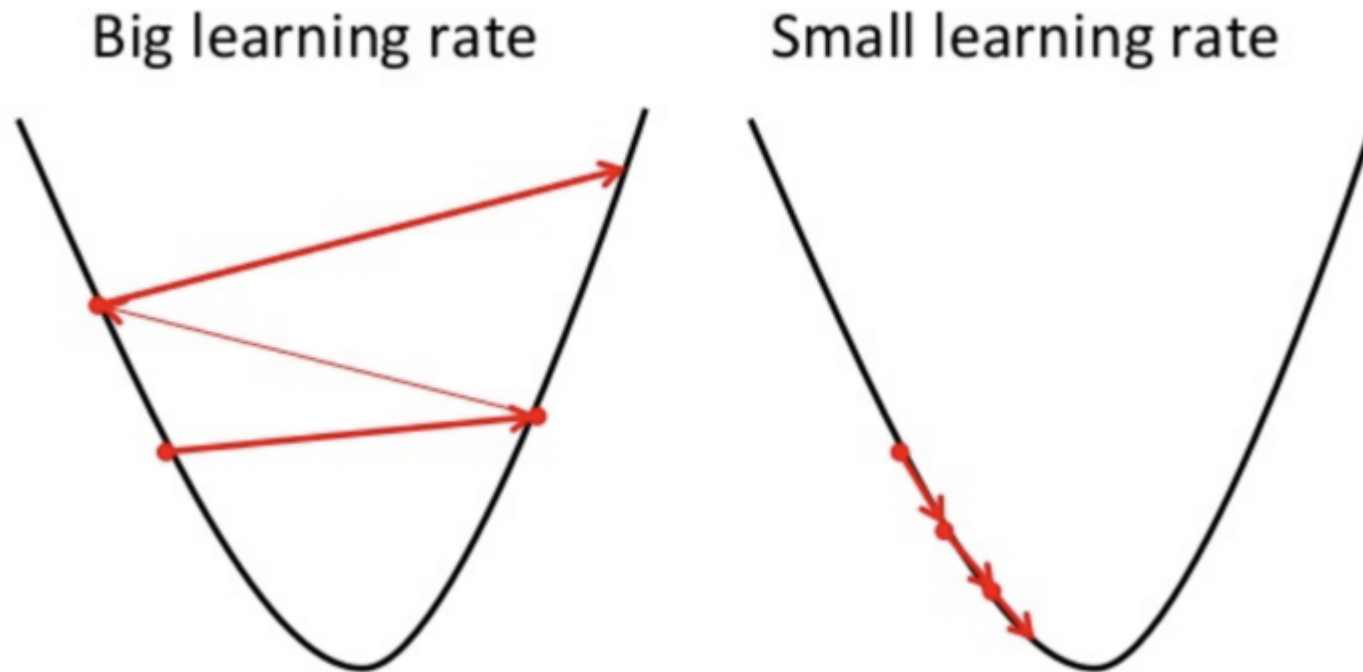
- It can be difficult to make a single step if:
 - $J(\theta)$ is poorly conditioned at the current point θ
 - θ lies on a cliff
 - θ is a saddle point hiding the opportunity to make progress downhill from the gradient
- It is possible to overcome all these problems and still perform poorly:
 - if the direction that makes most improvement locally does not point towards distant regions of much lower cost

Need for good initial points

- Optimization based on local downhill moves can fail if local surface does not point towards the global solution
- Research directions are aimed at finding good initial points for problems with a difficult global structure
 - For example: Trajectory of circumventing mountains may be long and result in excessive training time



Problem: The Learning Rate



Convergence of Steepest Descent

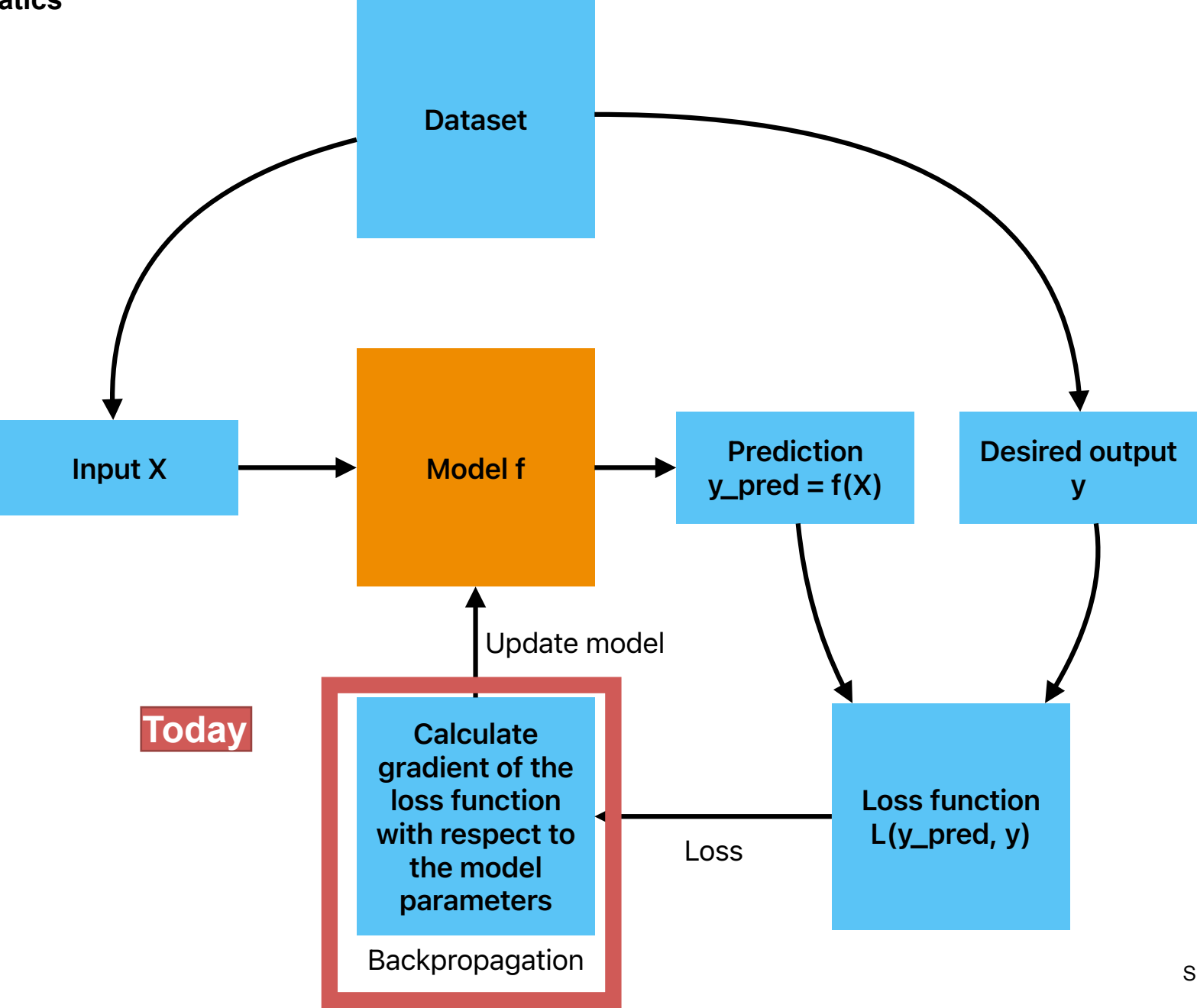
- Steepest descent converges when every element of the gradient is zero
- Pure math way of life:
 - Find literally the smallest value of $f(x)$
 - Or maybe: find some critical point of $f(x)$ where the value is locally smallest
- **Deep learning way of life:**
 - **Decrease the value of $f(x)$ a lot**
 - **But we have a highly non-convex problem (because of the activation functions) => No guarantees!**

About the Gradient

- Gradient must be large and predictable enough to serve as good guide to the learning algorithm
- Functions that **saturate** (become very flat) undermine this
 - Because the gradient becomes very small
 - Happens when activation functions producing output of hidden/output units saturate
- **Negative log-likelihood** helps avoid saturation problem for many models
 - Many output units involve exp functions that saturate when its argument is very negative
 - Log function in Negative log likelihood cost function undoes exp of some units

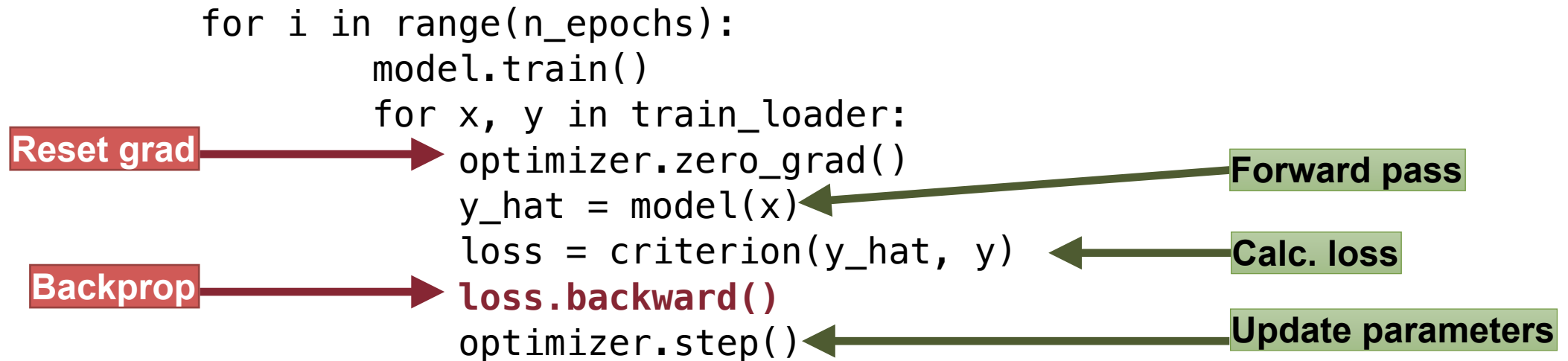
Keywords

- Gradient-based learning
- Stochastic gradient descent
- Challenges of optimization
- **Backpropagation**



Today

Backprop in the training loop



Backprop, 1986

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California,
San Diego, La Jolla, California 92093, USA

† Department of Computer Science, Carnegie-Mellon University,
Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.

Backprop, 1986

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

The backward pass starts by computing $\partial E / \partial y$ for each of the output units. Differentiating equation (3) for a particular case, c , and suppressing the index c gives

$$\partial E / \partial y_j = y_j - d_j \quad (4)$$

We can then apply the **chain rule** to compute $\partial E / \partial x_j$

$$\partial E / \partial x_j = \partial E / \partial y_j \cdot dy_j / dx_j$$

Chain Rule of Calculus

- If g is differentiable at x and f is differentiable at $g(x)$, then the composite function $F = f \circ g$ defined by $F(x) = f(g(x))$ is differentiable at x and F' is given by

$$F' = f'(g(x)) \cdot g'(x)$$

- In Leibnitz notation, if $y = f(u)$ and $u = g(x)$, then

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$$

A chain rule example

➤ $F(x) = (3x - 5)^2$

➤ Chain rule: $F(x) = f'(g(x)) \cdot g'(x)$

➤ Set $g(x) = 3x - 5$, and then the outer function is $f(g(x)) = g(x)^2$

➤ Calculate outer derivative $f'(g(x)) = 2 \cdot g(x)$, and multiply by inner derivative $g'(x) = 3$

➤ Substitute $g(x)$ to obtain $F'(x) = 2 \cdot (3x - 5) \cdot 3 = (6x - 10) \cdot 3 = 18x - 30$

Decomposing a neural net into functions

➤ Consider the function $f(\mathbf{x}) = \mathbf{W}^{(2)} \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x})$ representing a 2-layer neural net

➤ Can be decomposed into:

➤ $\mathbf{a} = \mathbf{W}^{(1)}\mathbf{x}$

➤ $\mathbf{z} = \text{ReLU}(\mathbf{a})$

➤ $\mathbf{y} = \mathbf{W}^{(2)}\mathbf{h}$

➤ And then we have the loss $E(f(\mathbf{x}), y)$

Or in function notation:

➤ $a(\mathbf{x}) = \mathbf{W}^{(1)}\mathbf{x}$

➤ $z(\mathbf{x}) = \text{ReLU}(\mathbf{x})$

➤ $y(\mathbf{x}) = \mathbf{W}^{(2)}\mathbf{x}$

➤ Then, $f = y \circ z \circ a$

➤ Or: $f(x) = y(z(a(x)))$

Differentiating the individual functions

➤ One hidden unit's weights: $a(\mathbf{z}) = \mathbf{w}^\top \mathbf{z} = w_1 z_1 + w_2 z_2 + \dots + w_n z_n$

$$\text{➤ } \frac{da}{dw_i} = z_i \quad \frac{da}{dz_i} = w_i$$

➤ Rectified linear unit: $\text{ReLU}(x) = \max\{0, x\}$

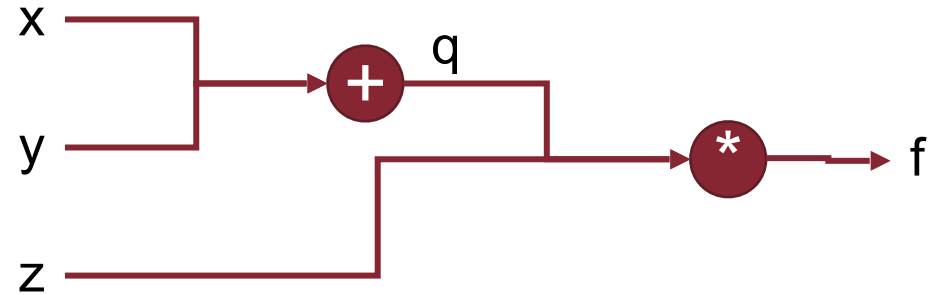
$$\text{➤ } \frac{d \text{ReLU}}{dx} := 1 \text{ if } x > 0 \text{ and } 0 \text{ otherwise (technically not differentiable at } 0 \text{ but we just define)}$$

➤ Loss function: $E(y, t) = \frac{1}{2} (y - t)^2$ (assuming 1d output)

$$\text{➤ } \frac{dE}{dy} = y - t$$

Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

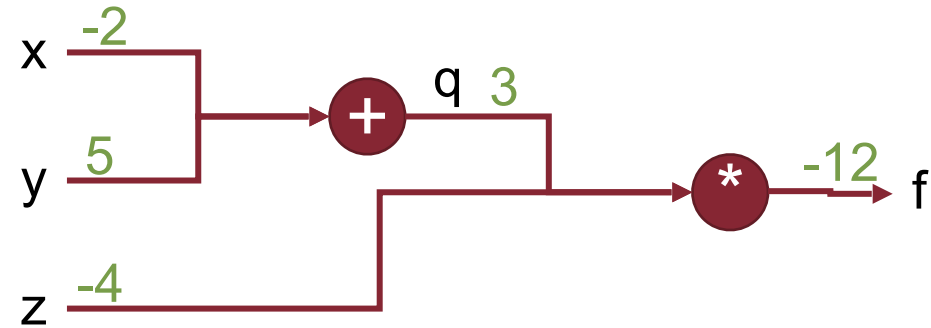


Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

➤ With

- $x = -2$
- $y = 5$
- $z = -4$



Backpropagation: A Simple Example

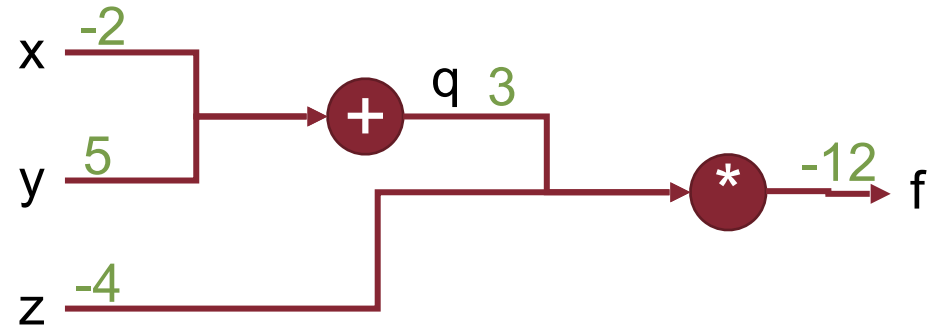
$$f(x, y, z) = (x + y)z$$

➤ With

- $x = -2$
- $y = 5$
- $z = -4$

$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$

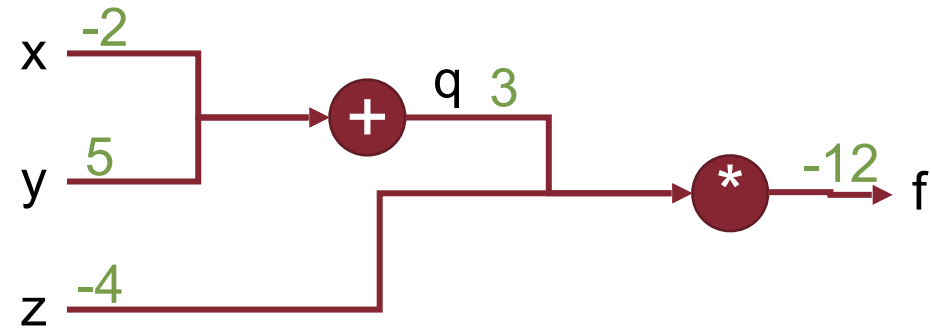


Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

➤ With

- $x = -2$
- $y = 5$
- $z = -4$



$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$

Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

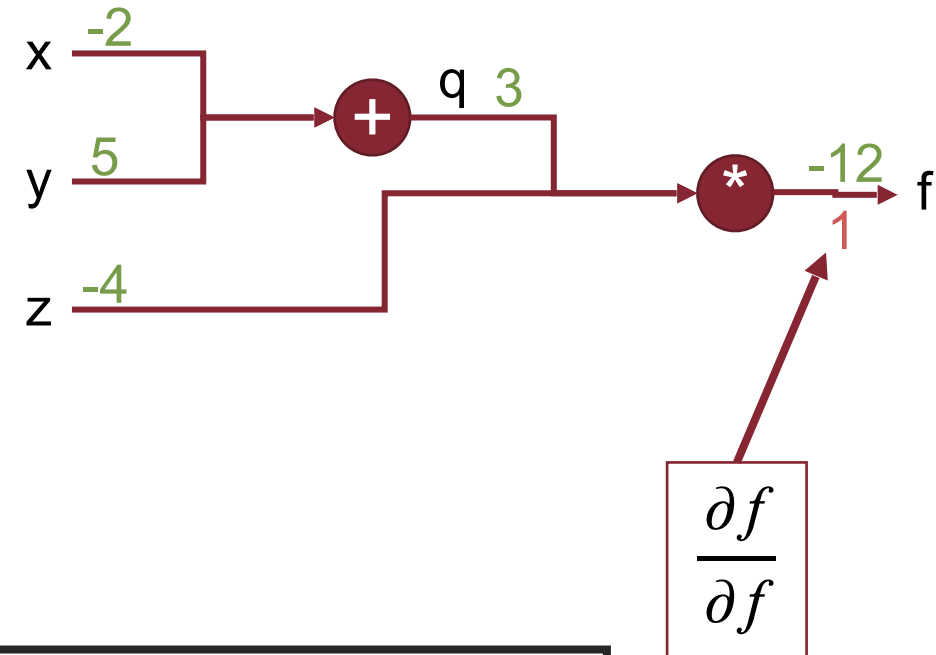
$$f(x, y, z) = (x + y)z$$

➤ With

- $x = -2$
- $y = 5$
- $z = -4$

$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$



Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

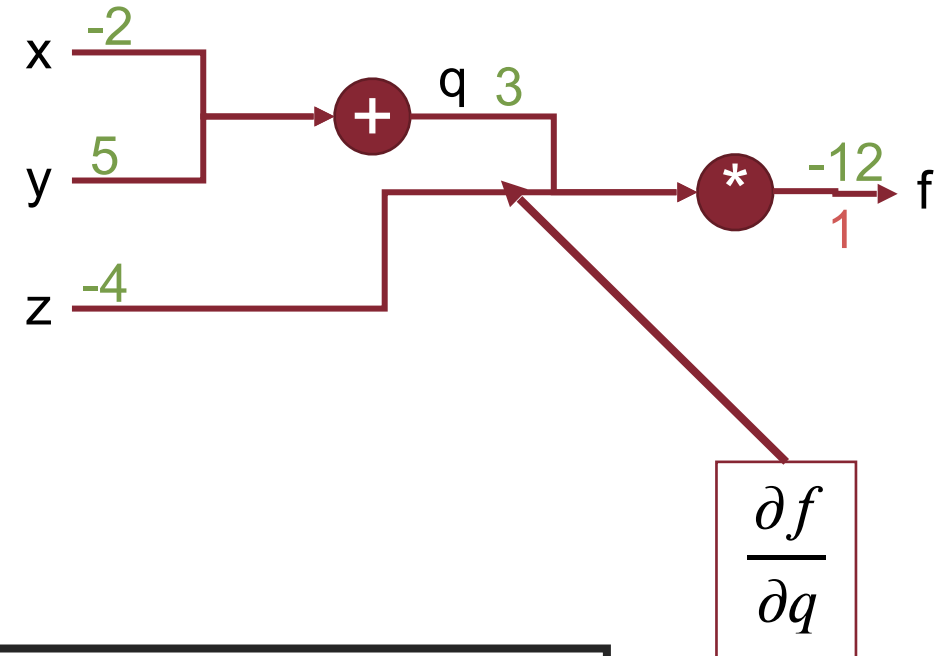
$$f(x, y, z) = (x + y)z$$

➤ With

- $x = -2$
- $y = 5$
- $z = -4$

$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$



Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

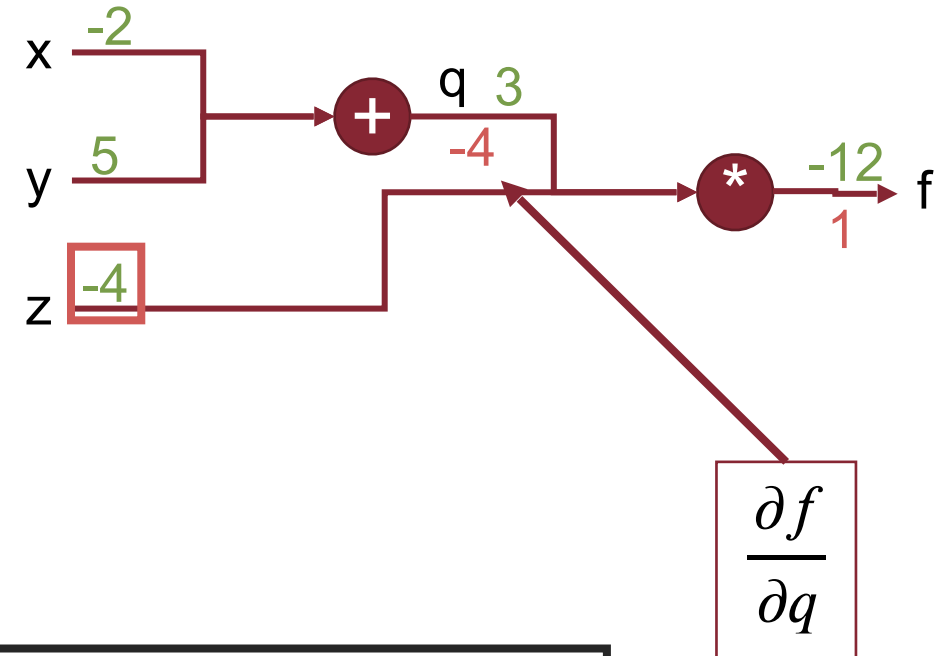
$$f(x, y, z) = (x + y)z$$

➤ With

- $x = -2$
- $y = 5$
- $z = -4$

$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$



Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

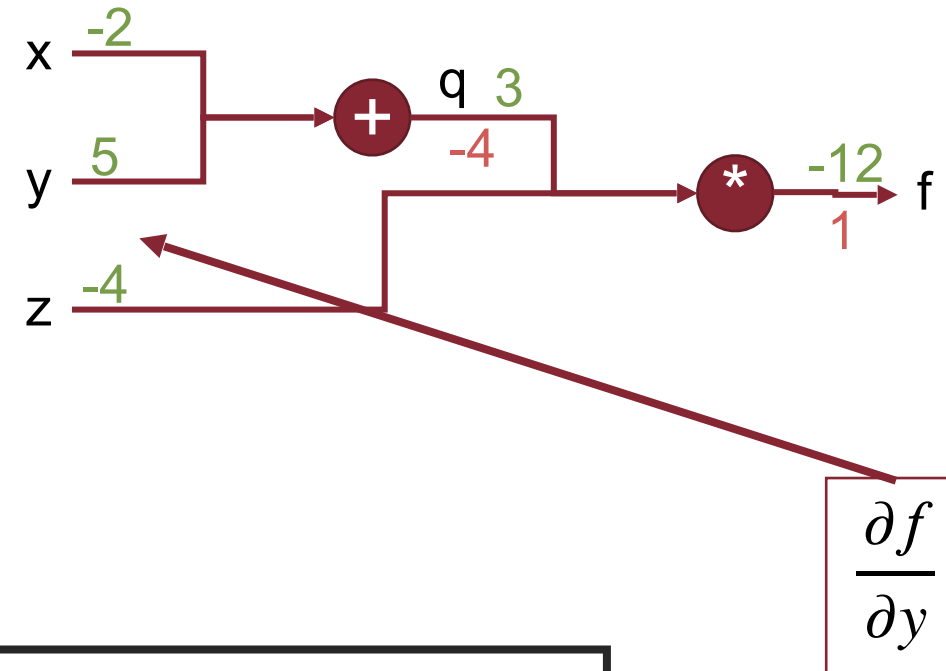
$$f(x, y, z) = (x + y)z$$

➤ With

- $x = -2$
- $y = 5$
- $z = -4$

$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$



Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

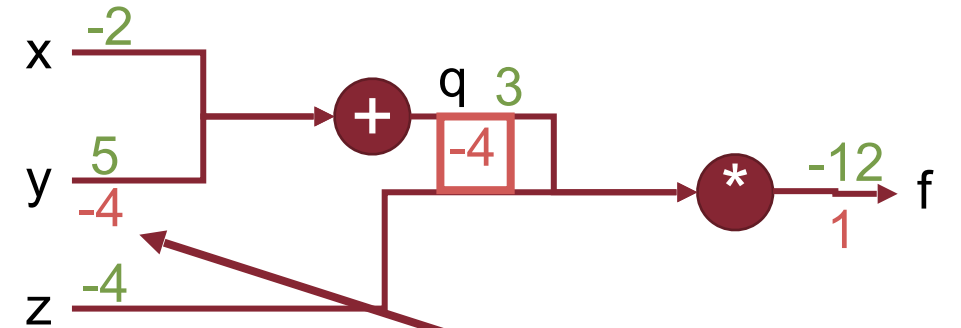
$$f(x, y, z) = (x + y)z$$

➤ With

- $x = -2$
- $y = 5$
- $z = -4$

$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$



Chain Rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

$$\frac{\partial f}{\partial y}$$

Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

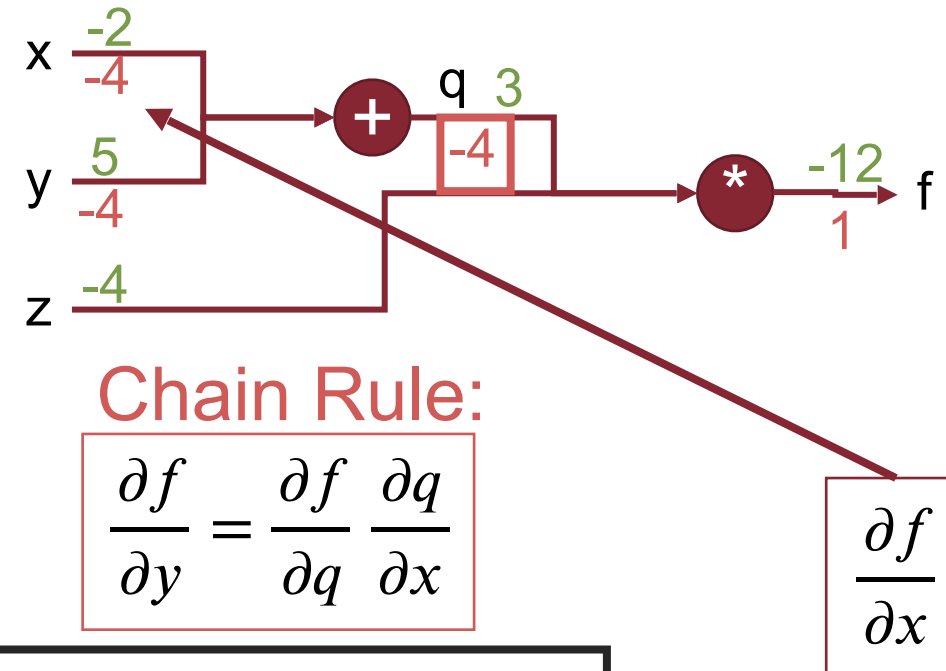
$$f(x, y, z) = (x + y)z$$

➤ With

- $x = -2$
- $y = 5$
- $z = -4$

$$q = x + y; \quad \boxed{\frac{\partial q}{\partial x} = 1}; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$



Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

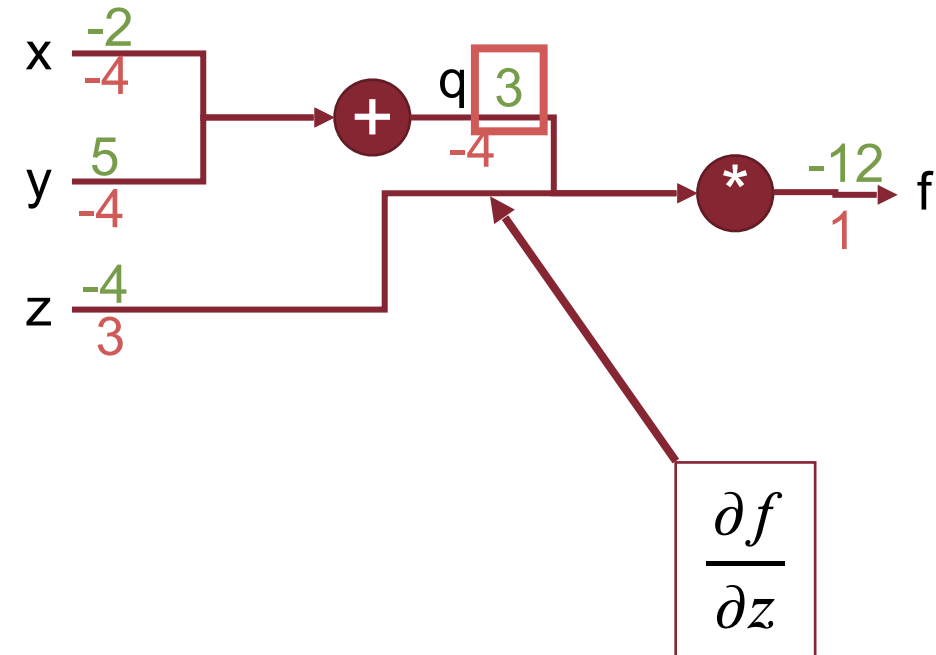
$$f(x, y, z) = (x + y)z$$

➤ With

- $x = -2$
- $y = 5$
- $z = -4$

$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$



Looking for:

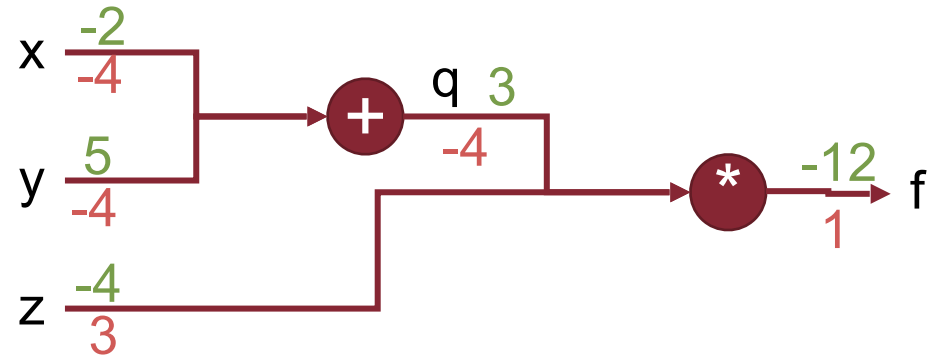
$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

➤ With

- $x = -2$
- $y = 5$
- $z = -4$



$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

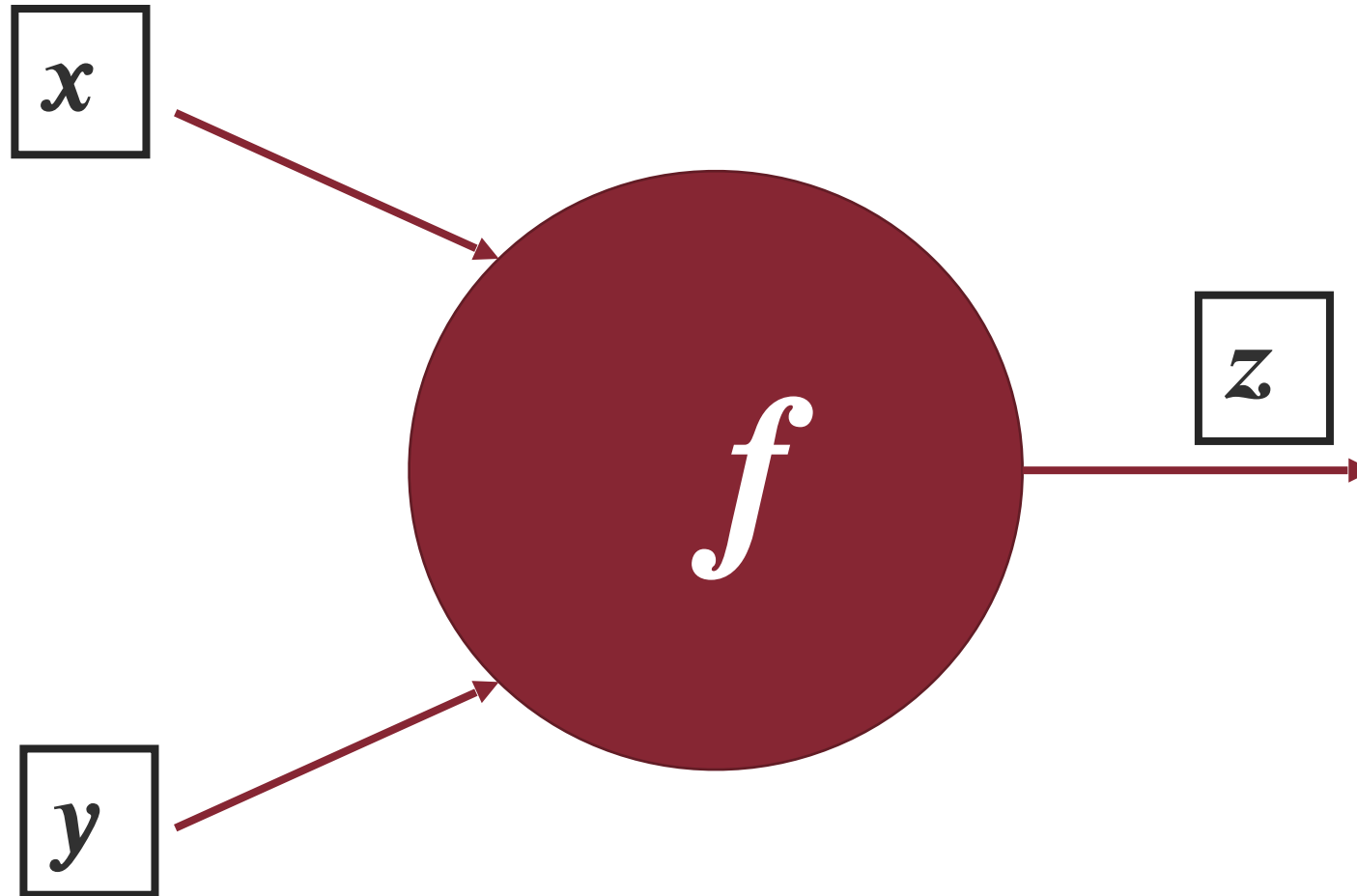
$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$

Looking for:

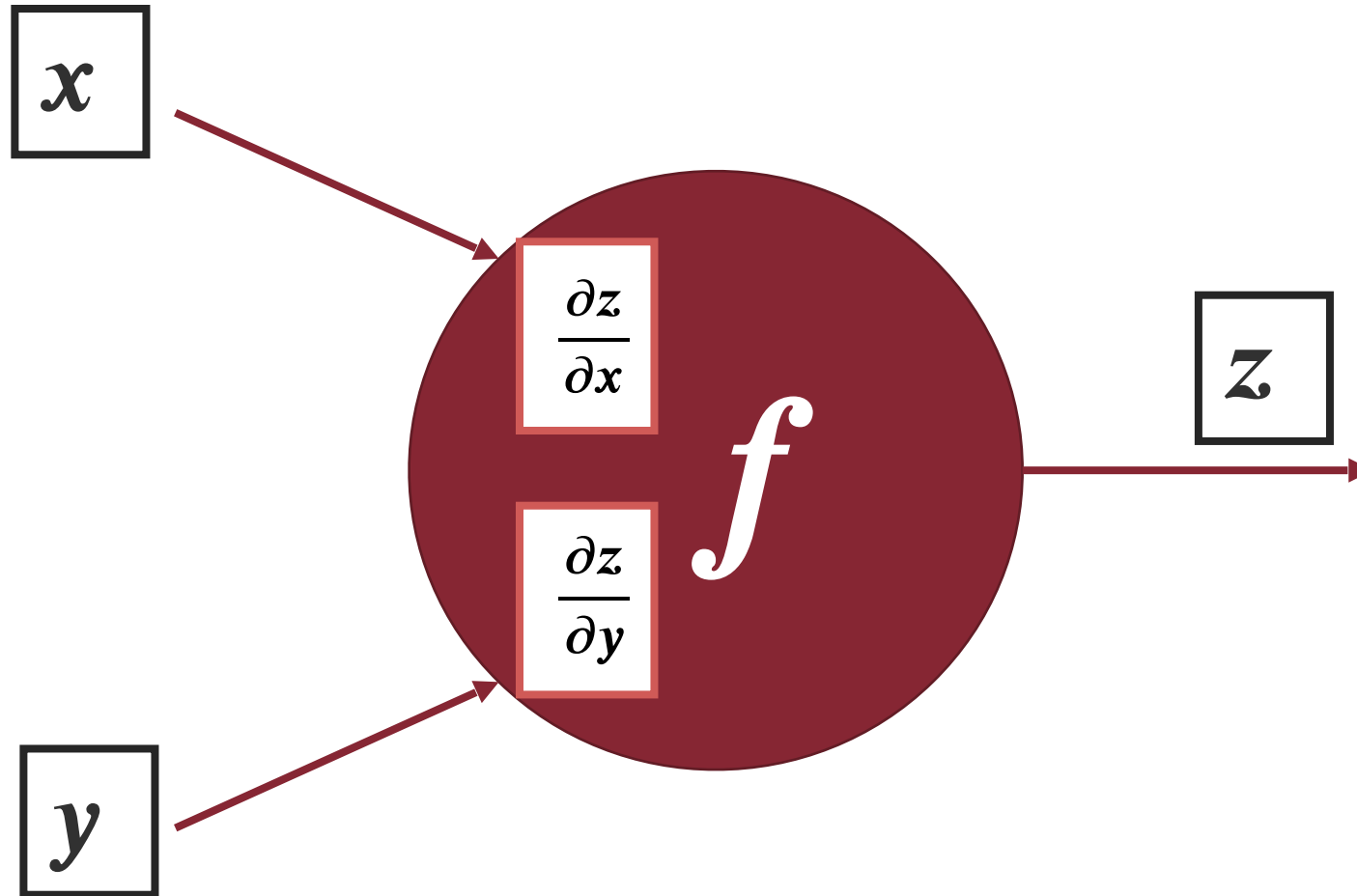
$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

$$\nabla_{x,y,z} f = \begin{pmatrix} -4 \\ -4 \\ 3 \end{pmatrix}$$

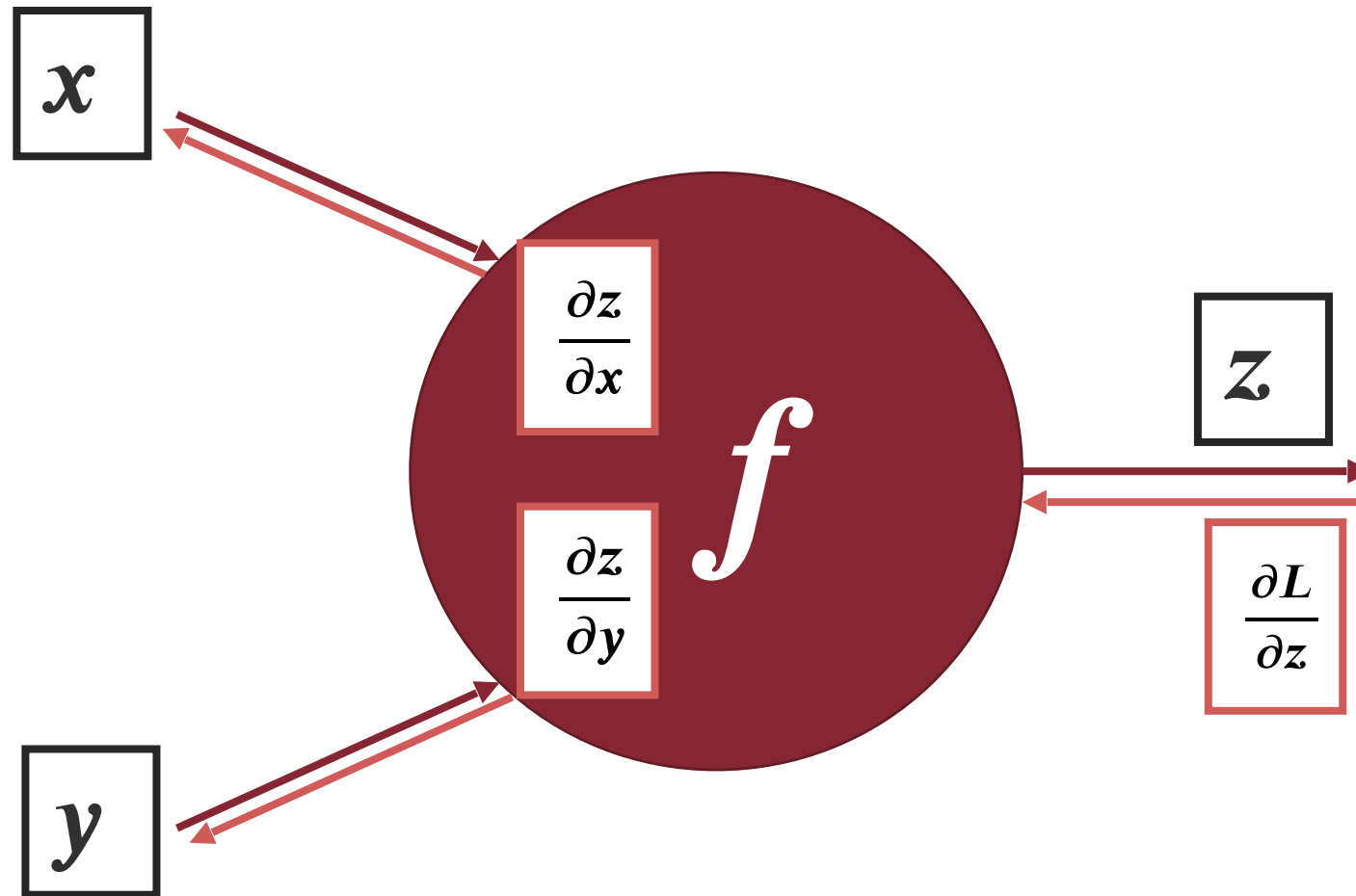
General Principle



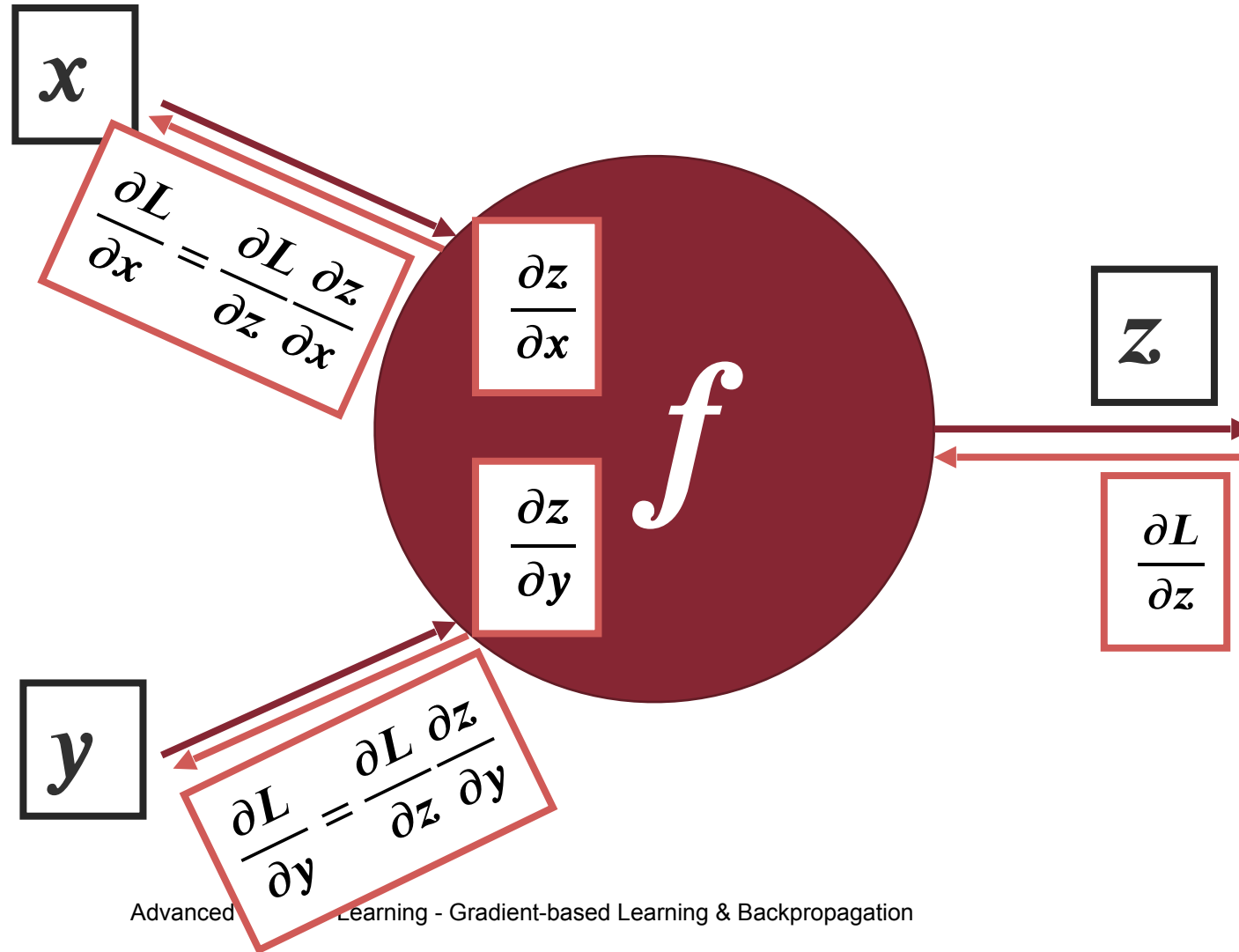
General Principle



General Principle



General Principle



Single-layer neural networks

- Linear model with multiple inputs x_i and multiple outputs y_i

$$y_k = \sum_i w_{ki} \cdot x_i$$

- Sum-of-squares error function for one data point n

$$E_n = \frac{1}{2} \sum_k (y_{nk} - t_{nk})^2$$

- Gradient

$$\frac{dE_n}{dw_{ji}} = (y_{nj} - t_{nj}) \cdot x_{ni}$$

Error signal for connection between
input i and output j

There is already a chain rule application here:

$$\frac{dy_k}{dw_{ki}} = x_i$$

$$\frac{dE_n}{dy_{nk}} = y_{nk} - t_{nk}$$

$$\frac{dE_n}{dw_{ij}} = \frac{dE_n}{dy_{nj}} \cdot \frac{dy_{nj}}{dw_{ji}}$$

General feedforward networks

Set of units, each computing the weighted sum of its inputs:

$$a_j = \sum_i w_{ji} z_i$$

Then we feed this pre-activation into a nonlinear activation function

$$z_j = h(a_j)$$

We assume that the forward pass (=feeding the data through the network) has already happened, and we have all the activations and the loss

Applying chain rule for partial derivatives gives

$$\frac{\partial E_n}{\partial w_{ij}} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}}$$

Useful notation for the **'errors'**: $\delta_j = \frac{\partial E_n}{\partial a_j}$

Now we know that:

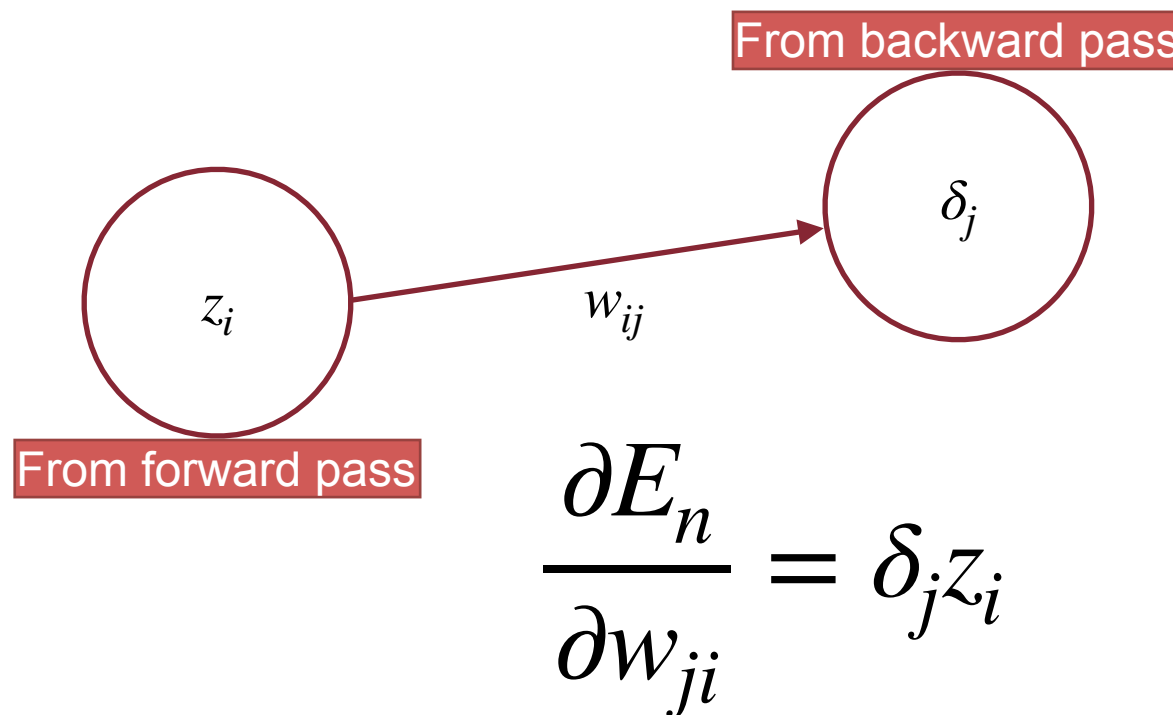
$$\frac{\partial a_j}{\partial w_{ji}} = z_i$$

And by substitution we get

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$$

From errors to derivatives for weights

- When we have the error δ_j for a hidden/output unit, we can derive the partial derivatives for its inbound weights by multiplying the error at the end of the connection with the activation at the start.



General feedforward networks (cont.)

- For the final layer, we get the error from the loss function – just as in the single layer case:

$$\delta_k = y_k - t_k$$

- To get the deltas for the hidden units, apply chain rule

$$\delta_j = \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j}$$

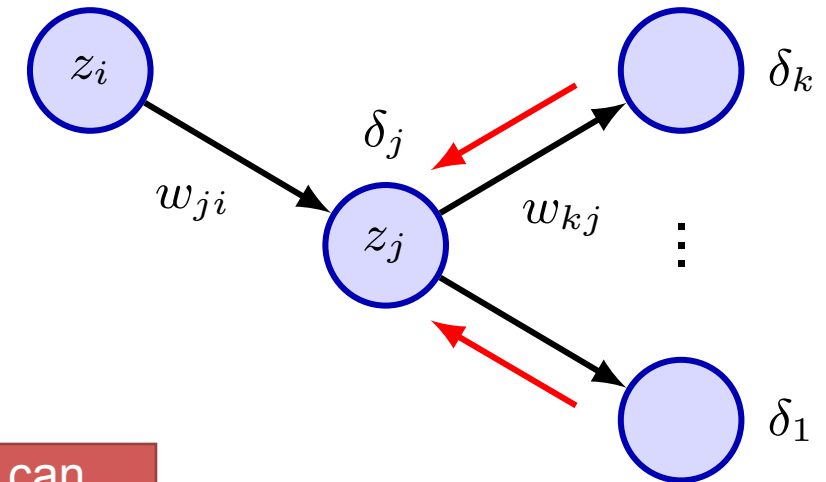
The sum runs over all units k to which j sends something

- Now substitute definition of $\delta_k = \frac{\partial E_n}{\partial a_k}$ in there

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

This tells us how we can propagate the errors back, starting from the output

Derivative of
activation function



Backprop in a feedforward network

- Consider the network

$$y = W^{(2)} \tanh(W^{(1)}x)$$

- And a mean-squared error loss function

$$E_n = \frac{1}{2} \sum_{k=1}^K (y_k - t_k)^2$$

- Forward pass:

$$a_j = \sum_{i=0}^D w_{ji} x_i$$

$$z_j = \tanh(a_j)$$

$$y_k = \sum_{j=0}^M w_{kj}^{(2)} z_j$$

Derivative of tanh

$$\frac{\partial z_j}{\partial a_j} = (1 - z_j^2)$$

- Get output error

$$\delta_k = y_k - t_k$$

- Backpropagate errors

$$\delta_j = (1 - z_j^2) \sum_{k=1}^K w_{kj}^{(2)} \delta_k$$

- Calculate the derivatives

$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j x_i$$

$$\frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k z_j$$

Backprop in a feedforward network with ReLU

- Consider the network

$$y = W^{(2)} \text{ReLU} (W^{(1)}x)$$

- And a mean-squared error loss function

$$E_n = \frac{1}{2} \sum_{k=1}^K (y_k - t_k)^2$$

- Forward pass:

$$a_j = \sum_{i=0}^D w_{ji} x_i$$

$$z_j = \text{ReLU}(a_j)$$

$$y_k = \sum_{j=0}^M w_{kj}^{(2)} z_j$$

Derivative of ReLU

$$\frac{\partial z_j}{\partial a_j} = \begin{cases} 1 & \text{if } a_j > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Get output error

$$\delta_k = y_k - t_k$$

- Backpropagate errors

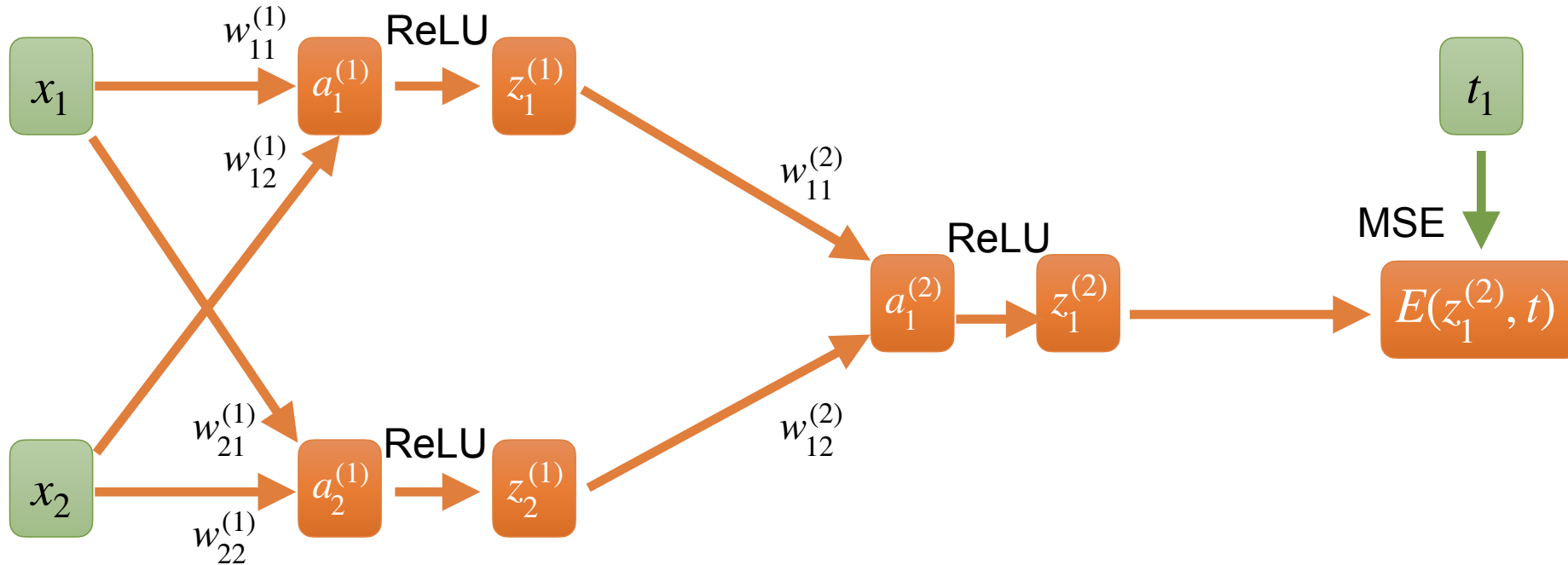
$$\delta_j = \begin{cases} \sum_{k=1}^K w_{kj}^{(2)} \delta_k & \text{if } a_j > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Calculate the derivatives

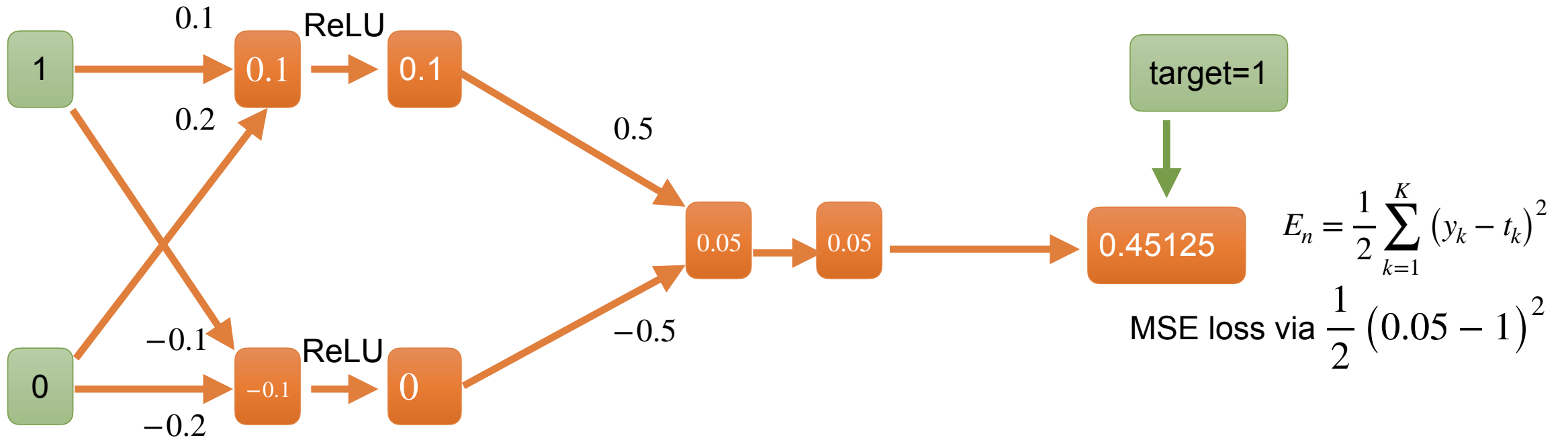
$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j x_i$$

$$\frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k z_j$$

Let's try it out: 2 inputs, 2 hidden units, 1 output

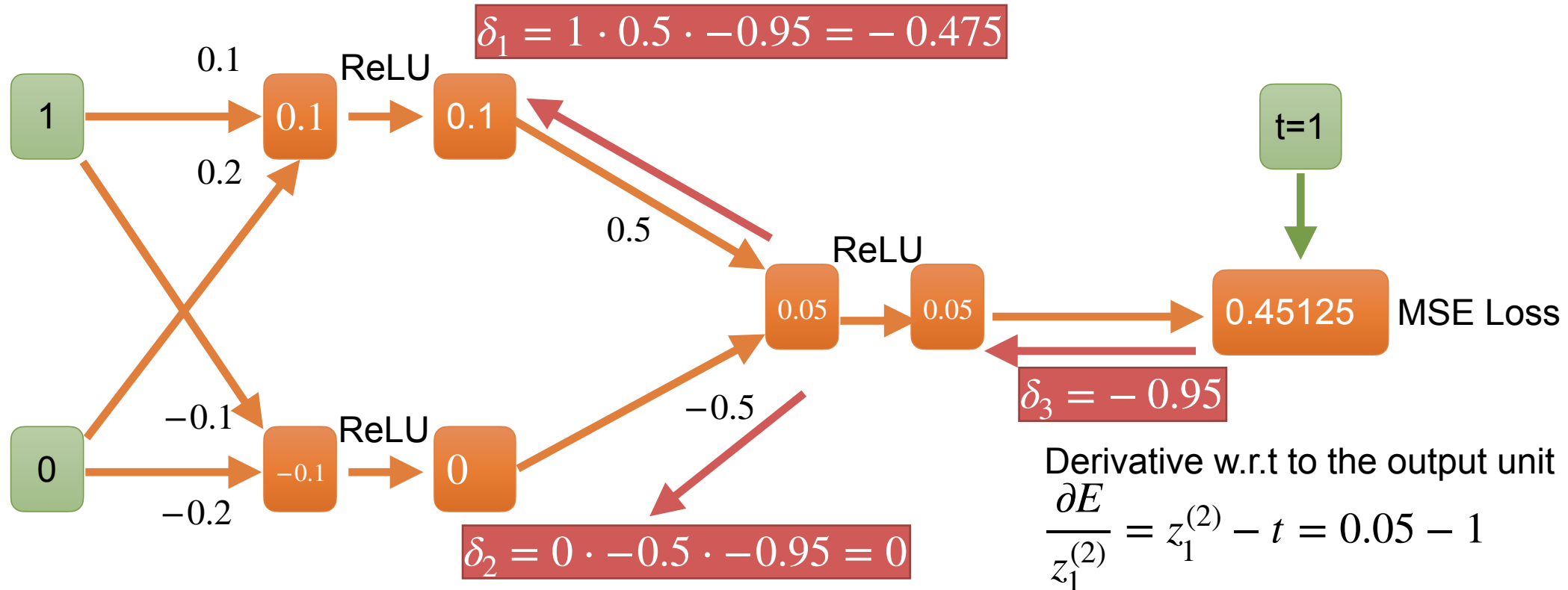


Example forward pass



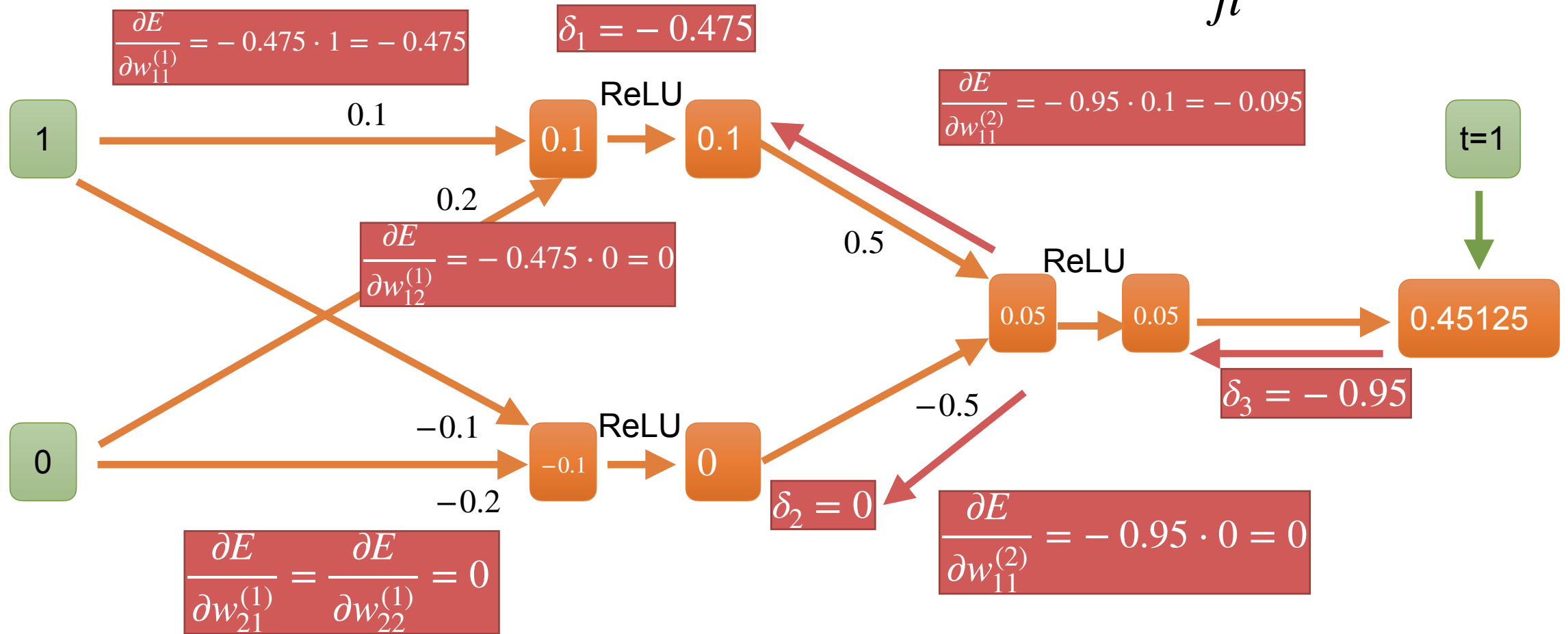
Example backward pass

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$



Example gradient

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$$



Backprop Algorithm

Algorithm 8.1: Backpropagation

Input: Input vector $\mathbf{x}_n$

Network parameters $\mathbf{w}$

Error function $E_n(\mathbf{w})$ for input x_n

Activation function $h(a)$

Output: Error function derivatives $\{\partial E_n / \partial w_{ji}\}$

```
// Forward propagation
for  $j \in$  all hidden and output units do
     $a_j \leftarrow \sum_i w_{ji} z_i$  //  $\{z_i\}$  includes inputs  $\{x_i\}$ 
     $z_j \leftarrow h(a_j)$  // activation function
end for

// Error evaluation
for  $k \in$  all output units do
     $\delta_k \leftarrow \frac{\partial E_n}{\partial a_k}$  // compute errors
end for

// Backward propagation, in reverse order
for  $j \in$  all hidden units do
     $\delta_j \leftarrow h'(a_j) \sum_k w_{kj} \delta_k$  // recursive backward evaluation
     $\frac{\partial E_n}{\partial w_{ji}} \leftarrow \delta_j z_i$  // evaluate derivatives
end for

return  $\left\{ \frac{\partial E_n}{\partial w_{ji}} \right\}$ 
```

Notes on backprop

- How to deal with multiple examples in a batch?
 - Sum the error up (textbook) or average the error over examples (default in practice)
- Backprop is not a learning algorithm by itself.
 - The purpose of backprop is to calculate gradients.
 - Needs to be coupled with a learning algorithm, e.g. stochastic gradient descent
- Backprop is not specific to deep learning, it can be used elsewhere.
- Generally, there are alternatives to backprop to calc. gradients (e.g. forward accumulation)
 - But backprop is most suitable for deep learning (because high number of parameters)

Keywords

- Gradient-based learning
- Stochastic gradient descent
- Challenges of optimization
- Backpropagation

How's my teaching?



<https://www.admonymous.co/lukasgalke>